

Time-Series Analysis:

Foundations, Models, and Evaluation

Abstract

Time-series analysis is organised around a single substantive assumption—stationarity—and a single methodological one—causality. This chapter develops the subject from those two commitments. It begins with the nature of time-indexed data and the generating processes behind it, establishes stationarity and the ways it fails, and proceeds through the practical work of representing, exploring, decomposing and cleaning temporal data. On that foundation it builds the three modelling families in current use: classical statistical forecasting, machine-learning forecasting over engineered features, and deep sequence models. It then treats the unsupervised problems—anomaly detection, change-point detection and clustering—which share the same distance measures and decomposition machinery, and closes with the evaluation protocols that make any of these results trustworthy. Terminology is defined at first use, and formulas are stated together with their interpretation and their conditions of validity.

1 Foundations

A **time series** is a sequence of observations indexed by time. **Regular** series carry observations at fixed intervals—hourly temperature, monthly sales—while **irregular** series carry them only when an event occurs, as with a patient’s laboratory results, recorded solely on clinic visits. Nearly every method that follows assumes regularity, which is why enforcing a regular index is among the first operations performed on raw data (Section 3).

The applications span most of quantitative practice: stock-price forecasting, electricity-demand prediction, traffic-flow prediction, weather forecasting (including wave height, earthquake and flood forecasts), sales forecasting, disease-outbreak prediction, website-traffic prediction, predictive maintenance for machinery, audio and speech signal prediction, and player-behaviour modelling in games. Forecasting is not the only task. A useful taxonomy of temporal problems also includes trend and seasonality detection, anomaly detection, change-point and early-warning systems, causal analysis, similarity search, denoising, and clustering of series. Sections 8–12 develop forecasting; Section 11 takes up the rest.

Behind every series lies a **data-generating process (DGP)**. A **deterministic** DGP evolves by a fixed rule—radioactive decay, in which the remaining quantity follows an exact formula and diminishes consistently. A **stochastic** DGP evolves randomly but with structure, as daily weather does. Interesting forecasting problems are almost always stochastic: fully deterministic series require no statistics, and fully random ones admit no forecast.

1.1 Synthetic building blocks

The DGP idea is best approached constructively, by generating series from a small set of primitives that can be mixed and matched to produce arbitrary complexity. These primitives are precisely the components that decomposition later attempts to recover.

White noise is a sequence of independent random values with zero mean and constant variance. It is the standard model of "no signal" and the standard assumption about residuals. **Red noise** shares the zero mean and constant variance but is *serially correlated*, generated recursively as

$$v_t = r v_{t-1} + \sqrt{1-r^2} w_t,$$

where w_t is a white-noise sample and r is the correlation coefficient parameterising the "redness". The factor $\sqrt{1-r^2}$ is what preserves constant variance while introducing memory.

Cyclical or **seasonal** signals are built from sinusoids; a **pseudo-periodic** variant randomises frequency and amplitude slightly, reproducing the imperfect periodicity of real data. An **autoregressive (AR) signal** makes the current value depend on previous values, parameterised by the *order* of serial correlation—how many past steps matter—and the *coefficients* that weight them. Serial correlation is the defining property of AR signals and the property that makes forecasting possible at all.

1.2 What can be forecast

The basic assumption underlying forecasting is that the future depends on the past. Series differ enormously in how far that assumption carries. Ranking next Monday’s high tide, next Sunday’s lottery numbers and a given stock’s price next Friday by predictability makes the point: the tide is nearly deterministic, the lottery is pure noise, and the stock sits awkwardly in between. Share prices are not a function of their own past but of *anticipated future* values, and they carry a very low signal-to-noise ratio. The **efficient market hypothesis (EMH)** holds that prices already reflect all publicly known information, so that any predictable change is already built into the price. The hypothesis is not literally true, but its failure is not large enough to make stock forecasting easy.

Three factors govern predictability. The better the DGP is understood, the more predictable the series. The more data available, the better. And there must be an *adequately repeating pattern*: the more repeatable it is, the better the forecast. The third factor is the binding constraint for most mathematical models, since without repetition there is nothing to learn.

2 Stationarity

A series is **stationary** when its probability distribution does not change over time: choose any two windows and the data distribution across them is the same. Since a Gaussian distribution is determined by its mean and variance, stationarity can break in exactly two ways.

Change in mean is the common case. An upward or downward **trend** makes the mean of an early window differ from that of a late one. **Seasonality** produces the same effect at shorter range: mean winter temperature and mean summer temperature are not equal, so a

monthly temperature series is non-stationary even in the complete absence of trend. This deserves emphasis, because it is a frequent source of confusion—seasonality is a violation of stationarity, not merely a nuisance pattern superimposed on a stationary process.

Change in variance is heteroscedasticity: fluctuations grow, or shrink, as time passes. The Air Passengers series—monthly international airline passenger totals—is the canonical example, its oscillations widening year on year.

Stationarity matters because the classical apparatus depends on it: most classical models, the autocorrelation function, the discrete Fourier transform, and the cost functions of change-point detection all assume it. Several remedies recur.

Decomposition estimates the trend and seasonal components and subtracts them, leaving a residual closer to stationary (Section 5). *Differencing* models changes rather than levels,

$$x'[n] = x[n] - x[n - 1],$$

which removes trend and stabilises the mean; the derivative of a signal generally has better stationarity properties than the signal itself. Differencing carries a warning: if analysis is performed on the differenced signal, the result must be re-integrated afterwards, and that re-integration is a source of error. *Framing* is the cruder alternative—divide the signal into short windows on which stationarity is plausible and analyse each separately. Finally, when changes are abrupt rather than smooth, their locations may themselves carry the information of interest, which is the subject of change-point detection (Section 11.2).

3 Representing temporal data

3.1 Datetime handling and core operations

Dates arrive as strings and must be converted. In the standard Python stack, `pd.to_datetime` returns a `Timestamp` for a single string and a `DatetimeIndex` for a list, and the resulting `datetime64[ns]` dtype unlocks the remainder of the time-series machinery. Automatic parsing fails predictably in two situations: ambiguous day/month ordering—month-first is assumed, so 4 January 1987 is misread unless day-first parsing is requested—and non-standard layouts, which require an explicit `strptime` format string (`%d` for zero-padded day, `%B` for full month name, `%Y` for four-digit year). With a datetime index in place, the `.dt` accessor exposes calendar properties such as `month`, `day_of_week` and `day_of_year`; `pd.date_range` generates sequences from a start, an end and a frequency; and `pd.Timedelta` offsets dates by days, months or other units. Making the date column the index further enables slicing along the temporal axis.

Four operations are easily confused and are worth stating side by side. **Resampling** changes the temporal frequency, aggregating (downsampling) or disaggregating (upsampling). **Shifting** moves values along the time axis to create lags or leads. A **rolling window** computes a statistic over a fixed-length recent window. An **expanding window** computes it over all data seen

so far. Rolling and expanding differ only in whether the window length is fixed or growing; shifting and resampling differ in whether the index itself changes.

3.2 Layout, metadata, and scale

A realistic dataset contains three kinds of information. **Time-series identifiers** name a particular series—an SKU, a household ID, a location. **Static features**, or metadata, do not vary with time, such as a household’s demographic classification. **Time-varying features**, such as weather, take a different value at every timestamp.

Three storage layouts follow from this. **Wide** format uses the date as index and gives each series its own column. It is traditional but scales badly: the table grows a column per series, it cannot hold per-series metadata, and it maps poorly onto relational databases, since new series require new columns. **Compact** format stores an entire series in a single row, with metadata in ordinary columns and the time-varying values held as arrays, plus `start_datetime` and `frequency` columns from which the timeline is reconstructed; it is memory-efficient for regularly sampled data. **Expanded**, or **long**, format gives each time step its own row, repeating identifiers and metadata, with an explicit timestamp column; it is the most flexible and the most database-friendly.

The size difference is not marginal. In a representative half-hourly energy dataset—consumption readings for 5,567 households over roughly two and a quarter years, with demographic classification, tariff type (dynamic time-of-use versus flat rate), and weather and bank-holiday enrichment—one block of data occupies about 0.002 MB compact against about 47 MB expanded.

Raw sources are rarely in either form. In this case each row holds a date while the columns hold 48 half-hourly blocks, so conversion requires first finding the global end date across all block files, then reshaping the half-hourly columns into long form. Static household metadata is then attached by a left join on the identifier. Time-varying tables—weather, holidays—must instead be aligned exactly: convert the date column to datetime, set it as the index, resample to the series frequency, forward-fill within each day, and fill any residue with an explicit sentinel such as `NO_HOLIDAY`, so that every timestamp carries a value. Only then can the aligned values be extracted as arrays and stored alongside the corresponding series.

Even nominally regular series contain gaps, so **enforcing regular intervals** is mandatory. Build a standard date frame running from each series’ start date to the global end date, left-merge the series onto it, and leave genuine gaps as `NaN` for later imputation. Series that terminate early—a household that leaves the programme—are handled by the same mechanism.

Storage has one further asymmetry worth anticipating. A merged compact frame occupying only about 10 MB in memory can expand to roughly 15 GB when serialised, because of the arrays held in its columns; CSV fails outright for the same reason. One option is a **time-series text format**: a text file augmented with the

column names, column types and metadata needed to read it back. The file on disk is still large, but because it is written and read as a streamed I/O operation, that volume never has to be resident in memory. The generally preferable option is to split the frame into chunks and write them as **parquet**, which compresses well, allows partial reads for quick experimentation, and preserves dtypes across the write/read cycle.

4 Exploratory analysis

4.1 Relationships between variables

Correlations among variables must be examined for two distinct reasons. **Feature leakage** occurs when a variable unintentionally gives away the target, producing results that look excellent and generalise not at all. **Collinearity** is correlation among the predictors themselves, which destabilises linear models; the remedy is to drop one of the correlated pair or to reduce dimension, for instance by principal component analysis.

The **Pearson correlation coefficient** between variables X and Y is

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y},$$

where the covariance is the expected product of the deviations from the means,

$$\text{Cov}(X,Y) = \mathbb{E}[(X - \bar{x})(Y - \bar{y})],$$

so that in sample form

$$\hat{\rho}_{X,Y} = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sigma_x \sigma_y}.$$

It measures *linear* association only. Because of the squared terms in the standard deviations it is sensitive to outliers, and it fails when the relationship is non-linear or non-continuous. The **Spearman rank correlation** applies the same formula to the *ranks* of the values rather than the values themselves, which makes it robust to outliers and to monotonic non-linear relationships—though not to non-continuous ones, since numerical differences are discarded and only ordering counts.

An air-pollution series makes the distinction concrete and doubles as a general warning. Over the full record, the Spearman correlation of suspended particulate matter with time is -0.19 , suggesting almost no relationship. Split at 1880, the two halves give $+0.99$ and -0.99 respectively, because within each period the ordering of points aligns nearly perfectly. A single correlation computed across a regime change is not merely imprecise; it is actively misleading.

Scatter plots are the visual counterpart to these coefficients, plotting two numerical variables against each other, optionally combined with the marginal histograms—so that the density of points along each axis is visible—and with a best-fit line superimposed to show how one variable behaves as a function of the other.

4.2 Trend, seasonality, and autocorrelation

Three characteristics organise nearly all descriptive work. A **trend** is a long-term increase or decrease in the

sequence. **Seasonality** is variation recurring at fixed regular intervals of less than a year, and may operate on several scales at once—daily, weekly, monthly, yearly. **Cyclic variation** consists of rises and falls that are *not* of fixed frequency. The distinction between the last two is precisely the fixity of the period, and conflating them is a standard error.

Autocorrelation is the correlation of a signal with a lagged copy of itself. The **autocorrelation plot (ACF)** draws it as a function of lag and is the primary tool for detecting repeating patterns and periodic signals. The **partial autocorrelation function (PACF)** isolates the correlation at each lag after removing the effect of shorter lags. ACF and PACF together drive order selection for models in the ARMA family (Section 8).

An ACF that fails to decay towards zero is itself evidence of non-stationarity. For the pollution series, a non-flat autocorrelation combined with a visible trend—and hence a changing mean—confirms that the process is not stationary.

Supporting displays include seasonal box plots, one box per period, which expose seasonal spread as well as seasonal level; calendar heatmaps; and resampling to a coarser frequency to see through short-term noise. Electroencephalography data illustrates a further use: brain activity decomposes into several superimposed oscillations at distinct frequency bands, and frequency-domain views separate them.

5 Decomposition

Seasonal decomposition deconstructs a series into trend, seasonality and residual under one of two models:

$$\begin{aligned} \text{additive: } y_t &= \text{trend}_t + \text{season}_t + \text{noise}_t, \\ \text{multiplicative: } y_t &= \text{trend}_t \times \text{season}_t \times \text{noise}_t. \end{aligned}$$

The choice is usually made visually. Use the additive model when the seasonal effect is approximately a fixed *amount*; use the multiplicative model when it is proportional to the level of the series. The Air Passengers series, whose oscillations widen with its trend, is multiplicative.

The general procedure has two stages. **Detrending** estimates the trend—the smooth change in the series—and removes it, yielding a detrended series. **Deseasonalising** estimates the seasonal component from the detrended series and removes it; what remains is the residual.

5.1 Estimating the trend

A **moving average** can be seen as a window slid along the series in steps, recording the mean of the values in the window at each step. It estimates slow change well. Its defect is noise: even after smoothing, the extracted trend is not smooth, and noise ideally belongs in the residual rather than in the trend.

LOESS—locally estimated scatterplot smoothing, i.e. locally weighted polynomial regression—is a non-parametric alternative that fits a smooth curve to a noisy signal. Treating an ordinal time variable as independent and the series as dependent, it estimates, at each point, a weighted regression using only a fraction

of the nearest points, with the highest weight at the point in question and weights decaying with distance. The result is a genuinely smooth trend estimate, and it is why STL-family methods handle complex trends better than moving averages do.

5.2 Estimating the seasonality

Period-adjusted averages compute a seasonality index for each position in the expected cycle by averaging the values at that position across all cycles. For monthly data with yearly seasonality the cycle length is 12: average all January values, all February values, and so on. Given the illustrative data

	Yr 1	Yr 2	Yr 3
Jan	100	110	120 $\Rightarrow$ 110
Feb	90	95	100 $\Rightarrow$ 95
Mar	130	135	140 $\Rightarrow$ 135

each period average is then compared with the average of all period averages. Writing j for the period,

additive $\text{index}_j = \text{period avg}_j - \text{overall avg}$,

multiplicative $\text{index}_j = \text{period avg}_j / \text{overall avg}$.

If the overall average is 100 and the January average is 110, the additive January index is $110 - 100 = 10$ —January runs 10 units above average—while the multiplicative index is $110/100 = 1.10$, or 10% above average.

A **Fourier series** represents a periodic pattern as a combination of sine and cosine waves, which suits seasonality exactly because seasonality is periodic. Writing P for the cycle length ($P = 12$ for monthly data with yearly seasonality) and N for the number of Fourier terms, small N produces simple, smooth seasonality while large N produces more complex seasonality at increasing risk of overfitting. N is therefore a complexity dial. Sine and cosine terms are complementary—each pair captures one harmonic at arbitrary phase—which is why they always appear together.

5.3 Four implementations

Method	Trend	Seasonal	Multi	NaN
Seasonal decomp.	Mov. avg	Period avg	No	No
STL	LOESS	Period avg	No	Yes
Fourier decomp.	LOESS	Fourier	No	No
MSTL	LOESS	Period/Fourier	Yes	No

Seasonal decomposition in its basic form (`seasonal_decompose` in `statsmodels.tsa.seasonal`) estimates the trend by moving averages and the seasonality by period-based averages. Its parameters are `period`, the length of the seasonal cycle; `model`, additive or multiplicative; `filt`, controlling the moving-average window; `extrapolate_trend`, which fills trend values at the series edges; and `two_sided`, which selects between a centred window using past and future values and a backward-looking window using only the past. The last matters whenever the decomposition must respect causality. Missing values are not supported, so imputation must come first.

STL—seasonality and trend decomposition using LOESS—replaces the moving average with LOESS, making it markedly more flexible for complex trends, and

retains period-adjusted averages for the seasonality. Beyond `seasonality_period` and `model` it adds `lo_frac`, the fraction of data used in each local fit (default 0.6), and `lo_delta`, a fractional distance within which linear interpolation replaces full weighted regression (default 0.01), which is purely a speed optimisation.

Fourier decomposition pairs a LOESS trend with Fourier seasonality controlled by `n_fourier_terms`. The seasonality may be specified through a datetime property such as `week_of_day` or `month`, or supplied directly as an ordinal array the same length as the series—for annual seasonality of daily data, an array running from 1 to 365 and incrementing each day.

MSTL extends STL to several seasonal periods simultaneously, which is necessary for high-frequency data carrying hourly, weekly and yearly cycles at once. Its value is not cosmetic: seasonalities that are not extracted remain in the residual, contaminating every subsequent analysis.

6 Cleaning: outliers and missing values

6.1 Detecting outliers

An **outlier** is an unusual data point differing so strongly from the rest that it may have been generated by a different mechanism. Outliers arise from errors, faulty measurements or rare events. Four detectors are standard, sharing one important caveat: for series with trend or seasonality they should be applied to the *residuals after decomposition*, not to the raw values, or ordinary seasonal peaks will be flagged.

The **standard-deviation rule** marks values beyond $\mu \pm 3\sigma$ as outliers. Under a normal distribution roughly 99% of values fall inside this range, so those outside are rare by construction. The multiplier is adjustable—larger multiples detect fewer outliers—and the rule relies on approximate normality.

The **interquartile range (IQR)** rule uses quartiles instead of moments. With $\text{IQR} = Q_3 - Q_1$, outliers are the values outside

$$[Q_1 - 1.5 \text{IQR}, Q_3 + 1.5 \text{IQR}].$$

Because quartiles are barely affected by extreme values, this is more robust than the standard-deviation rule.

Isolation Forest is an unsupervised method that uses random decision trees to isolate unusual points. Outliers are easier to isolate and therefore appear in shorter branches, while normal points require longer paths; the anomaly score reflects how quickly a point is isolated. A `contamination` parameter sets the expected proportion of anomalies. Its limitation is fundamental in this setting: it ignores time order entirely and detects only values that look unusual in the marginal distribution.

The **extreme studentized deviate (ESD)** test, based on Grubbs’s test, assumes normally distributed data and repeatedly identifies and removes one outlier at a time, updating the critical threshold as it goes. **Seasonal ESD (S-ESD)** extends this to seasonal data by first removing the seasonality. Its practical advantage over the simpler rules is that it requires only an upper bound on the expected number of outliers, rather than

a manually tuned threshold, and then identifies them automatically.

Treatment should be conservative. Detected outliers must be *verified* before correction, because they may represent meaningful patterns rather than errors. Small datasets warrant manual inspection; at scale, outliers are replaced by a heuristic value or treated as missing and imputed. Correction is optional—modern machine-learning and deep-learning models often tolerate outliers—so its effect should be established experimentally rather than assumed.

6.2 Missing values

Missingness should never be handled reflexively by dropping rows or filling with the mean. The first question is what the DGP implies. A supermarket product with no transactions on a given day usually means *zero sales*, not a missing value, and the gap should be filled with zero. A systematic gap—no data every Sunday—can safely be zero-filled only if the model also receives a day-of-week feature, so that it recognises the pattern instead of learning a spurious weekly drop. And a product that normally sells well suddenly showing zero may indicate a genuine data problem or a special event: a point-of-sale failure, an entry error, or a stock-out. Each calls for careful imputation rather than literal interpretation.

Imputation itself divides into **unit imputation**, which replaces missing values with constants such as 0, the mean or the median, and **model-based imputation**, which predicts them from other variables. Unit imputation is simpler and more widely used. The concrete techniques are:

- **Forward fill** (last observation carried forward): take the last observed value and use it until the next observation appears.
- **Backward fill** (next observation carried backward): take the next observation and backtrack over the gap.
- **Mean fill**: use the mean of the entire series.
- **Linear interpolation**: draw a line between the two bracketing observations and place the missing values on it.
- **Nearest interpolation**: use the closer of the two neighbouring observations—intuitively a combination of forward and backward fill.
- **Spline and polynomial interpolation**: fit a curve of specified **order** through the observed points. Higher order means a more flexible—and more erratic—fit.

For long gaps in seasonal data a **seasonal profile** works better: compute the hourly average, or the hourly average for each weekday, and impute from it. Its weakness is that it ignores trend completely. **Seasonal interpolation** repairs this in three steps: compute the seasonal profile; subtract it and apply any of the interpolation techniques above to the deseasonalised series, which still carries the trend; then add the seasonal profile back.

7 Transformations and feature construction

7.1 Feature transforms

Feature transforms modify raw features so that they better match model assumptions. Many classical models work best on approximately normal, comparably scaled inputs, and transforms can reduce skewness, stabilise variance and make features comparable.

Min-max scaling maps values into a fixed range, usually $[0, 1]$, preserving relative order while changing scale. **Z-score normalisation** produces mean 0 and standard deviation 1, and converts Gaussian data into standard normal data. Both matter most for models sensitive to feature magnitude—linear models and distance-based methods.

Neither, however, changes the *shape* of the distribution. For that there is the **log transformation**, which compresses large values and reduces skew: values in $(0, 1)$ map to $(-\infty, 0)$ while values ≥ 1 are compressed. It suits log-normally distributed features. **Power transformations** generalise it, seeking a parameter λ that makes the data as normal as possible while preserving monotonicity. **Box–Cox** is the standard choice; it requires strictly positive values and reduces exactly to the log transformation at $\lambda = 0$. **Yeo–Johnson** extends it to zero and negative values.

These two are the workhorses of temporal preprocessing for a specific reason. Trend and seasonality are handled by differencing and decomposition; power transformations address what remains, the *dispersion structure* of the data—variance instability over time and departure from normality. By compressing high values and stretching low ones in an optimal way, they stabilise variance, which improves both point accuracy and, more importantly, the reliability of forecast confidence intervals. And λ is found automatically, typically by maximum-likelihood estimation, eliminating manual trial and error.

The **quantile transformation** is a more aggressive alternative. It maps a feature through an estimate of its cumulative distribution function onto a uniform distribution, optionally then onto a normal one. Its advantage is that features measured on entirely different scales become convenient to process, plot and compare. Whichever transform is used, the data should be inspected before and after, and normality tested rather than assumed.

7.2 Feature engineering

Feature engineering creates new features that help a model learn better patterns. The goal is features that are predictive, informative, and not unnecessarily redundant. **Hand-crafted** features are designed from domain knowledge and are typically interpretable—means, maxima, minima, trends, rolling statistics—and are preferred wherever explainability matters. **Automated** extraction generates large numbers of descriptors mechanically, through libraries and algorithms such as **tsfresh**, **featuretools**, **Catch22**, **HCTSA**, **ROCKET** and **shapelets**.

One constraint governs all of it. Temporal features must use only past and present information. Using future data creates **leakage** and yields overly optimistic results that evaporate in deployment. A valid pipeline respects the prediction time at every step—which is why centred rolling windows, two-sided decomposition, and any statistic computed over the full record are quiet hazards.

Six feature groups are conventional: date-related, time-related, calendar, window-based, convolutional, and shapelet features. The calendar family repays detailed attention because it is cheap and often decisive.

Holiday features matter wherever behaviour is calendar-driven—sales, traffic, finance, web activity—and country- or region-specific calendars can be generated programmatically, for instance with **workalendar**. The profile of people applying for a loan on a bank holiday differs from that on a weekday, and a model given the holiday flag can represent the difference. **Date annotation** extracts the day of week, the number of days since the start of the year, the number until its end, and the analogous within-month quantities. **Payday features** encode that many people are paid mid-month or on the last Friday of the month, which propagates directly into consumer demand. **Season features** capture cyclic behaviour across the year. **Sun and moon features**—daylight hours, sunrise, dusk, moon phase—help where activity tracks daylight, since more daylight may correlate with more business activity or mobility. **Business-day counts** affect any monthly aggregate: a month containing 22 business days and 9 weekend days and holidays behaves differently from one with a different split, which matters for sales, operations, finance and demand forecasting.

7.3 ROCKET

ROCKET—Random Convolutional Kernel Transform—transfers the success of convolutional networks into a preprocessing step. A **convolution** applies a kernel over local parts of the data; in images kernels sharpen or detect edges, and in series they detect local temporal patterns.

ROCKET applies a large number of *random* convolutional kernels—10,000 by default—to each series. Every kernel slides across the series and produces a response sequence showing how strongly its pattern matches at each location. Rather than retain the sequence, ROCKET summarises it with two scalars. The **maximum** captures the strongest occurrence of the pattern, answering "how strongly did this kernel's pattern appear anywhere?". The **proportion of positive values (PPV)** captures how frequently it appears, answering "in how many parts of the series did this kernel produce a positive response?". For a response $[-0.5, 4.5, 0, 1.5]$, the maximum is 4.5 and $PPV = 2/4 = 0.5$.

One kernel therefore reduces an entire series to two numbers, and 1,000 kernels produce 2,000 features per series. These are fed to a simple linear model—ridge regression or logistic regression are the recommended choices. Two differences from a convolutional neural network explain why ROCKET is fast and needs no

training: it uses no hidden layers and no non-linearities, and its convolutions are applied independently to each feature channel. The kernels are never learned; all discriminative work is done by the linear model above them.

7.4 Shapelets

Shapelets are discriminative subsequences—local patterns that distinguish between classes. The contrast with global alignment is the essential idea. To separate healthy from unhealthy electrocardiogram signals, a global alignment method such as dynamic time warping compares the two waveforms over their entire length. A shapelet approach instead searches for a small, specific dip or spike appearing only in the unhealthy signals, and once that subsequence is identified, classifies a new series by a simple rule: how close is the nearest match of this shapelet within the new series?

That "within" gives shapelets **shift invariance**: the model looks for the shape and is indifferent to whether it appeared at the beginning, middle or end. Shapelets are also interpretable—the shapelet itself can be plotted and shown to a domain expert—fast at prediction time once the dictionary is learned, and competitive on classification benchmarks.

8 Classical statistical forecasting

Classical forecasting models are grounded in statistical theory, widely used in econometrics, designed primarily for univariate series, effective when data is limited, and interpretable. They are also, almost always, linear and dependent on stationarity, requiring preprocessing such as detrending or differencing to enforce it. Real-world series are frequently non-linear and non-stationary, which is the opening for the machine-learning and deep-learning methods of Sections 10 and 12. The standard workflow is: test for stationarity; difference if necessary; fit the method and forecast; add the trend and seasonality back.

8.1 Moving average, autoregression, ARMA

Two distinct meanings of "moving average" must be kept apart. The *simple moving average* is the smoother of Section 5: an average of previous observations that damps short-term fluctuation and reveals long-term trend. The **MA(q) model** is something else:

$$x_t = \mu + \epsilon_t + \sum_{i=1}^q \phi_i \epsilon_{t-i},$$

where μ is the expectation of x_t (usually assumed zero), q is the number of lagged error terms, ϵ_t is current white noise, and ϵ_{t-i} are previous white-noise terms weighted by ϕ_i . The predictors here are *residuals*, or innovations—not previous observations. An MA(q) process is thus a stationary series generated by a finite memory of past shocks. In

$$x_t = \mu + \epsilon_t + 0.7 \epsilon_{t-1},$$

a shock today influences the series for exactly one further step and then disappears entirely.

The **AR(p) model** takes the complementary view, mak-

ing the current value depend on previous *values*:

$$x_t = c + \sum_{i=1}^p \phi_i x_{t-i} + \epsilon_t,$$

with p lags, a constant c , and autoregressive coefficients ϕ_i . Its theoretical warrant is **Wold's decomposition**, which established that a stationary series can be expressed as the sum of a deterministic autoregressive component and a stochastic noise component. AR models are used in two ways: applied to the original or differenced series as the primary forecasting engine, and applied to the *residuals* of another model to remove remaining autocorrelation and correct a base forecast.

ARMA(p, q) merges both dependency structures:

$$x_t = c + \epsilon_t + \sum_{i=1}^p \phi_i x_{t-i} + \sum_{i=1}^q \theta_i \epsilon_{t-i},$$

which is more flexible than either component alone and requires stationary data. Orders are set by inspecting the ACF and PACF plots for the lags at which correlation peaks.

A useful reference point before either model is the **seasonal naive forecast**, which predicts each value by the observation one full season earlier. It requires no fitting and is often hard to beat on strongly seasonal data, which is exactly why it serves as the baseline for scaled error metrics (Section 9).

8.2 ARIMA and SARIMA

ARIMA(p, d, q)—autoregressive integrated moving average—adds **integration**, a preprocessing step that makes the series stationary by **differencing**, subtracting immediate past values:

$$x_t - x_{t-1}.$$

Differencing removes trends, stabilises the mean and achieves stationarity; the parameter d specifies how many times it is applied. Instead of modelling the values themselves, one models the changes between consecutive values. ARIMA is the most widely used classical forecasting model.

Seasonal ARIMA, written

$$\text{ARIMA}(p, d, q)(P, D, Q)_m,$$

adds a seasonal counterpart. Here m denotes the number of periods in a season, and P , D and Q parameterise the autoregressive, integration and moving-average components of the seasonal part, giving seasonal autoregression (SAR) and seasonal moving-average (SMA) terms with their own distinct parameter sets alongside the non-seasonal p , d and q . It suits yearly, monthly and weekly patterns.

8.3 Model selection

Competing models differ in complexity and in forecasting accuracy, and the goal is the simplest model that explains the data well—which buys interpretability, reduced overfitting and improved generalisation. ARMA-family models are usually estimated by **maximum-likelihood estimation (MLE)**, and the two standard

criteria penalise the likelihood by parameter count:

$$\text{AIC} = 2k - 2 \ln L, \quad \text{BIC} = k \ln N - 2 \ln L,$$

where k is the number of parameters, L the likelihood and N the number of samples. Lower is better in both cases: a good model has few parameters but high log-likelihood. Specialised to ARIMA these become

$$\text{AIC} = 2(p + q) - 2l, \quad \text{BIC} = k \ln(N) - 2l,$$

where d is omitted because differencing introduces no additional estimated parameters. The difference between the criteria is that the $\ln N$ factor penalises complexity more heavily as the sample grows, so BIC selects more parsimonious models on long series. The same two criteria reappear, in the same role, in change-point detection (Section 11.2).

Assumptions should be validated before and after fitting. The augmented Dickey–Fuller (ADF) and KPSS tests assess stationarity; ACF and PACF guide order selection; the Ljung–Box test detects residual autocorrelation; and the ARCH-LM test detects remaining heteroscedasticity.

8.4 Exponential smoothing

Exponential smoothing smooths a series using an exponential window function, and can forecast series with trend and seasonality. Its premise is that recent observations matter more than older ones. **Simple exponential smoothing (SES)** is

$$s_t = \alpha x_t + (1 - \alpha)s_{t-1},$$

where α controls responsiveness: higher α tracks recent movement closely, lower α produces smoother forecasts. Expanding the recursion shows $(1 - \alpha)$ multiplied by itself repeatedly—this is the exponential term, and the reason old observations decay geometrically in influence. SES is fast and computationally efficient, effective for stationary series, and models neither trend nor seasonality.

The **Theta method** is simple exponential smoothing with drift. A trend is fitted as a linear regression line over time,

$$T_t = c + b_0(t - 1) + \epsilon_t,$$

with intercept c (the starting level), slope b_0 , time index $t - 1$ and residual ϵ_t . The forecast is that trend added to the SES forecast as a weighted sum:

$$\hat{X}_t = (1 - \alpha)T_t + \alpha \hat{X}_{t-1}.$$

The method decomposes trend and short-term dynamics and performed exceptionally well in forecasting competitions. Its lesson is durable: simple methods often outperform complex ones.

The **Holt–Winters method**, also called **triple exponential smoothing**, applies exponential smoothing three times—to a level L estimated at each time point, a trend component T , and a seasonality S with m seasons—which suits seasonal forecasting problems. In the multiplicative-seasonality form the series is de-seasonalised by dividing by the previous season's index,

x_t/S_{t-m} , and the previous trend is added to the previous level as $L_{t-1} + T_{t-1}$. The components update as

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1},$$

$$S_t = \gamma \frac{x_t}{L_t} + (1 - \gamma)S_{t-m}.$$

Each equation is itself an exponentially weighted average, with β and γ playing for trend and seasonality the role α plays for the level. The additive/multiplicative choice mirrors the decomposition case—additive when the seasonal effect remains constant, multiplicative when it grows with the magnitude of the trend—and is usually settled by visual inspection.

8.5 Volatility and multivariate models

ARIMA models focus on predicting means. **ARCH** models focus on predicting variance, allowing time-varying, heteroscedastic error terms by parameterising higher-order dependence between returns observed at varying frequencies. The residual variance at time t is modelled from the squared residuals at previous points,

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q \alpha_i \epsilon_{t-i}^2,$$

where q is the number of preceding time points the variance depends on. ARCH(q) can be determined using least squares.

GARCH—generalised ARCH—improves on this by modelling previous *variances* as well as previous errors, which is what allows it to capture the **volatility clustering** commonly observed in financial markets, where calm and turbulent periods arrive in runs. The formal distinction is that GARCH residuals come from an ARCH model rather than from an autoregressive model.

Vector autoregression (VAR) tracks relationships between several variables as they change over time, capturing the linear dependence of a series on a vector of values preceding the current timestamp and thereby generalising AR to the multivariate case:

$$x_t = c + B_1 x_{t-1} + \dots + B_p x_{t-p} + \epsilon_t,$$

where x_t is a vector and the B_i are coefficient matrices, so that each series depends on the recent past of all series. VAR assumes the error terms have mean 0 and are not serially correlated.

Implementations concentrate in `statsmodels`, on which most other classical forecasting libraries rely, exposing `AutoReg`, `ARIMA`, `ExponentialSmoothing` and `SimpleExpSmoothing`; `pmdarima`, `arch`, `sktime` and `darts` build further layers on top. A complete workflow runs: load the data, resample, check stationarity, analyse ACF and PACF, select the model order, train, generate forecasts, evaluate.

9 Evaluation and validation

If $f(x_t)$ is the model’s prediction for time step t and y_t the actual target value, the **forecast error**—equivalently prediction error or residual—is

$$e_t = y_t - f(x_t).$$

The standard metrics are

$$\text{MSE} = \frac{1}{N} \sum_{t=1}^N e_t^2, \quad \text{RMSE} = \sqrt{\text{MSE}},$$

$$\text{MAE} = \frac{1}{N} \sum_{t=1}^N |e_t|, \quad \text{MdAE} = \text{median}(|e_t|),$$

$$\text{MAPE} = \frac{1}{N} \sum_{t=1}^N \frac{|e_t|}{|y_t|},$$

$$\text{SMAPE} = \frac{1}{N} \sum_{t=1}^N \frac{|e_t|}{(|y_t| + |f(x_t)|)/2},$$

$$\text{NMSE} = \frac{\text{MSE}}{\sigma^2},$$

$$\text{RMSLE} = \sqrt{\frac{1}{N} \sum_{t=1}^N \left(\log(f(x_t) + 1) - \log(y_t + 1) \right)^2}.$$

Their differences are their reason for existing. Squared-error metrics punish large errors disproportionately; absolute-error metrics do not, and the median variant is robust to a handful of catastrophic points. Percentage metrics make errors comparable across series of different magnitude but diverge as $y_t \rightarrow 0$ and penalise over- and under-prediction asymmetrically—SMAPE’s symmetric denominator is the standard correction. NMSE normalises by the variance of the series, so a score is interpretable relative to the difficulty of the series. RMSLE penalises relative rather than absolute error and suits series spanning orders of magnitude. **MASE**, the mean absolute scaled error, divides error by that of a naive baseline, so values below 1 beat the baseline and values above it do not. **Forecast bias** reports systematic over- or under-prediction, which symmetric error metrics conceal. R^2 and correlation metrics complete the set. In every case the goal is to minimise error on *unseen* data.

That last word marks the sharpest divergence from ordinary supervised learning. Standard k -fold cross-validation is misleading for temporal data, because it trains on observations that come after those it tests on. Since the process evolves over time, temporal ordering must be preserved and future data must remain unseen during training, or evaluation becomes overly optimistic.

Walk-forward validation enforces this. Train on historical observations, test on the observations immediately following, move the training window forward, and repeat across multiple folds. Model parameters are adjusted entirely on training and validation data, and the final benchmark uses a test set more advanced in time than either. The result is a realistic—and usually much less flattering—performance estimate, and a reduced risk of overfitting.

10 Machine-learning forecasting

Machine learning becomes necessary where the classical assumptions fail: non-linear structure, many exogenous variables, large collections of related series. The general recipe converts forecasting into supervised regression by

engineering features—lags, calendar variables, rolling statistics—and then applying a standard learner. Feature engineering therefore remains critical rather than incidental, and the representation and transformation techniques of Sections 3 and 7 are prerequisites rather than preliminaries.

10.1 From linear models to boosting

Linear regression carries five assumptions, each of which temporal data tends to violate: the relationship between independent and dependent variables is linear; the errors are normally distributed; the variance of the errors is constant across the values of the independent variable; there is no autocorrelation in the errors; and there is little to no correlation between independent variables. Autocorrelated errors are the near-universal violation in time series, and the reason residual diagnostics matter.

Regularised linear regression adds a penalty to the residual sum of squares,

$$\text{RSS} = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda W,$$

where the weight-decay term W is either the L_1 norm, the sum of absolute coefficient values, or the L_2 norm, the sum of their squares:

$$W_{L_1} = \sum_{i=1}^k |\beta_i| \quad (\text{lasso}), \quad W_{L_2} = \sum_{i=1}^k \beta_i^2 \quad (\text{ridge}).$$

The L_1 penalty drives coefficients exactly to zero and so performs feature selection, which is valuable after automated extraction has produced thousands of candidates. The L_2 penalty shrinks coefficients smoothly and stabilises estimates under collinearity, the normal condition when lag features are present. The fitted coefficient vector doubles as a feature-importance readout.

Decision trees partition the feature space into M regions $P_1, \dots, P_M$ and predict a constant within each, capturing non-linearity and interactions without any transformation of the inputs; a single tree, however, is unstable. **Random forests** average many trees fitted to resampled data, reducing that variance. **Gradient boosting**, in its **XGBoost** implementation, instead builds trees *sequentially*, each new tree reducing the residual error of the ensemble so far. Applied to forecasting, XGBoost is trained on past values, together with engineered features, to predict future values, and works for univariate and multivariate series alike. The pipeline runs: feature extraction, data transformation, XGBoost regression, training on historical observations, prediction of future values, and performance evaluation.

Two pieces of engineering make such pipelines reusable. A **date feature transformer**, written to the standard transformer interface, automates calendar-feature extraction so that it drops into any pipeline. And a feature configuration object formalises the bookkeeping: it holds the date column, the target and original target, the continuous, categorical and boolean feature lists, index columns and exogenous features, and exposes a single method returning the tuple (features, target, original target) with flags controlling whether categorical

and exogenous features are included. The structure is necessary because continuous, categorical and boolean columns each require different preprocessing before entering a model.

A large multivariate panel illustrates the shape of the problem. In a global epidemiological dataset, one row is one location on one date, and the columns divide into identifier columns, case columns, death columns, hospital and ICU columns, testing columns, vaccination columns, and demographic and background columns. The identifier / static-feature / time-varying-feature classification of Section 3 maps onto it directly: the location identifies the series, the demographic and background columns are static metadata, and everything else varies with time.

10.2 Instance-based methods and DTW

k -nearest neighbours (kNN) is an instance-based learning algorithm: it finds similar historical examples and makes predictions from its nearest neighbours. Simple as it is, it is surprisingly competitive. Three settings define it—the number of neighbours k , the integration function that combines them (the average for regression, the most frequent value for classification), and the distance function used to find them.

For temporal data the third setting dominates, and **dynamic time warping (DTW)** is the natural choice. DTW measures similarity by warping the time axis to find the alignment giving the lowest total distance, so it handles temporal misalignment and sequences of different lengths, and often outperforms Euclidean distance. If one series peaks at time 20 and another at time 25, Euclidean distance reports them as dissimilar while DTW recognises the same shape merely shifted in time. kNN with DTW is a historically strong benchmark for classification; its drawback is cost, since standard DTW is $O(T^2)$ for two sequences of length T .

A robotics dataset illustrates the alternative, feature-based route to the same task. Each robot execution carries an identifier, a time column indexing steps within that execution, and six sensor channels—forces F_x, F_y, F_z and torques T_x, T_y, T_z —with one binary label per execution distinguishing normal execution from failure. The pipeline is: take the raw sensor series, group by execution identifier, extract statistical features from each sensor, train a classifier on the extracted features, and predict normal or failure. The general lesson is that a classification problem over variable-length multivariate sequences can be reduced to a fixed-width tabular problem by feature extraction—the same move that **ROCKET** and automated descriptor libraries make.

10.3 Structured forecasting frameworks

Silverkite is a hybrid forecasting framework designed for speed and accuracy at scale, and distributed as part of the broader **Greykite** library. It decomposes a series into distinct, human-readable components, transforms datetime data, autoregressive lags and user-defined variables into mathematical basis functions, and feeds those into standard regression models such as ridge regression. Its primary components are *growth and trend*, modelling long-term change as linear, quadratic or other

shapes; *seasonality*, breaking daily, weekly and monthly recurrence into Fourier series; *events and holidays*, capturing calendar effects; and *autoregression*, which improves shorter horizons—under about 30 days—by using $y_{t-1}, y_{t-2}, \dots$ directly.

The full pipeline runs from the raw series through the creation of time, event, holiday, lag and regressor features; representation of the series in terms of trend, seasonality, holidays and events; change-point detection; fitting of a machine-learning model; generation of forecasts and accuracy reports; and finally residual modelling to produce prediction intervals and detect anomalies. Silverkite is therefore a genuine hybrid: classical time-series ideas combined with machine-learning models and explicit uncertainty estimation.

The surrounding library adds templates for different forecasting regimes—short-term hourly forecasts, long-term daily forecasts—together with experimental plots, tuning templates and model customisation. Known anomalies can be marked and then ignored, adjusted or kept. Outliers can be detected automatically, and the best model selected from candidate configurations on the basis of past performance, so that instead of tuning each forecast manually one defines several candidate setups capturing different trend hypotheses. Interpretation is supported directly: holiday effects can be estimated, the impact of change points shown, and the contribution of specific features to the forecast explained. Speed is an explicit design goal—a seven-day forecast trained on more than eight years of daily data can be produced in a few seconds, with the full pipeline completing in under 45 seconds.

Related tooling includes **Kats** for ensemble forecasting, **tsfresh** for automated feature extraction, and **sktime** as a unified framework for temporal machine learning. Ensemble models improve robustness, and the overall workflow is common to all of them: collect the data, perform feature engineering, split using walk-forward validation, train candidate models, evaluate, and deploy the best performer.

11 Unsupervised analysis

Many series carry no labels, and the questions that matter are discovery questions: what patterns exist, what is unusual, what has changed, and what resembles what. Two fundamental questions organise the event-detection part of this. Were there significant changes in the data across time? And was there something new or unusual? Both are ill-posed until "significant" and "unusual" are given operational meaning, and the choice of meaning is exactly what distinguishes the algorithms.

Temporal data makes the problem harder than the tabular case. Observations depend on time, so the same pattern may carry different meanings at different points; and the data is typically noisy, high-dimensional, non-stationary and continuously evolving. Dimensionality reduction, wavelet analysis, filtering and Fourier decomposition are therefore applied as preprocessing to reduce noise and extract the important patterns before detection begins. Humans can often identify anomalies and change points visually, particularly with the whole

history in view; automatic detection is harder and requires algorithms that balance high detection accuracy against a low false-positive rate.

11.1 Anomaly detection

An **anomaly**, or **outlier**, is a data point deviating so significantly from other observations that it could have been generated by a different mechanism. Applications include fraud detection, equipment monitoring, cybersecurity and healthcare. Methods divide into univariate and multivariate, and into parametric and non-parametric. **Parametric** methods place an assumption on the underlying distribution—often Gaussian—through the choice of distribution parameters such as the arithmetic mean, and flag the points that deviate from the model’s assumptions.

The simplest criterion is the **z-score**:

$$\text{outlier}(\hat{\mu}, \hat{\sigma}) = \left\{ x_i : \frac{|x_i - \hat{\mu}|}{\hat{\sigma}} > \epsilon \right\},$$

where $\hat{\mu}$ and $\hat{\sigma}$ are the estimated mean and standard deviation of the series and ϵ is a threshold set by the desired confidence interval. Choosing $\epsilon = 2$ or 1.96 corresponds to a 95% interval, so that flagged points are those occurring 5% of the time or less.

The **Hampel filter**, or Hampel identifier, is the robust special case, replacing the sample mean by the median and the standard deviation by the median absolute deviation:

$$HF(x) = \left\{ x_i : \frac{|x_i - \text{median}(x)|}{1.4826 \text{MAD}(x)} > \epsilon \right\},$$

$$\text{MAD}(x) = \text{median}(|x_i - \text{median}(x)|).$$

The constant 1.4826 rescales the MAD so that it estimates the standard deviation of Gaussian data, which makes ϵ interpretable on the same scale as in the z-score rule.

Applied globally—for instance through histogram inspection of the whole record—these criteria detect only aberrant values. Applied in a sliding window they become **adaptive**, performing a local statistical test at each position:

$$|x[n] - \mu_n| > \lambda \sigma_n, \quad |x[n] - \text{med}_n| > \lambda \text{mad}_n,$$

where μ_n , σ_n , med_n and mad_n are computed locally around sample n . Under an i.i.d. Gaussian assumption, $\lambda = 1, 2, 3$ correspond to 68%, 95% and 99.7%. The gain is local context. The cost is that time information is still discarded: only the distribution of values within the window is used.

Three kinds of anomaly must be distinguished, and they demand different machinery. **Point anomalies** are single unusual observations—isolated samples with exceptionally large or small values. **Contextual anomalies** take values that are perfectly normal in general but wrong for their context; a mild temperature is unremarkable except in midwinter. **Collective anomalies** are groups of observations, each individually plausible, that together form an unusual pattern—bursts of outliers, or

segments that do not match what is usually observed. Simple threshold methods detect the first kind only.

Model-based methods are the ones that genuinely exploit temporal structure. The procedure has three steps: choose an adequate model and learn its parameters; compute the prediction or signal reconstruction; and flag the samples that diverge from the model. A trend-plus-seasonality model—for instance a degree-4 polynomial trend with cosine and sine functions at fundamental frequencies that are multiples of $1/12$ —works, as does an AR model of order p . Such methods detect not merely large or small values but aberrant *temporal progression*. One consequence deserves attention: with an AR model, an anomaly also corrupts the prediction of the next p samples, so a single event can generate a burst of flags.

Distance-based methods handle the hardest case, in which the anomaly is not an aberrant value at all but a behaviour not previously seen, definable only as a divergence from normal behaviour. These methods effectively cluster points into groups and label the small groups as outliers, so the choice of distance measure is crucial. They also handle much larger spaces than the low-dimensional or univariate statistical methods.

The **matrix profile** is the central tool here. Given a pattern length L ,

$$m[n] = \min_{i > n+L \text{ or } i < n-L} d(x[n:n+L-1], x[i:i+L-1])$$

records the distance from the subsequence beginning at n to its nearest non-trivial match elsewhere in the series, computed efficiently with a normalised Euclidean distance. *Small* values indicate that the subsequence recurs, marking it as a pattern or motif. *Large* values indicate a subsequence far from everything else—a discord, and a strong anomaly candidate, likely to correspond to a new behaviour. Pattern discovery and anomaly detection are thus dual problems solved by one computation: instead of searching for repetitive patterns, one searches for non-repetitive events. The matrix profile requires no parametric model, but does require a rough idea of the anomaly’s scale through L .

A related approach divides the signal into possibly overlapping subsequences, clusters them with k -means or spectral clustering, and treats subsequences far from their centroids or medoids as outliers. **Semi-supervised** approaches instead use knowledge of normality: learn a model of normal behaviour and detect anomalies in the residual, or match annotated normal templates against the signal up to a measure of fit and flag every segment corresponding to no known pattern. Fully **supervised** classification—support vector machines, random forests, neural networks—is available where labels exist.

Mature implementations are plentiful: Prophet, which places uncertainty intervals around the estimated trend component; PyOD, with some 30 detection algorithms for multivariate data; alibi-detect, offering likelihood ratios, spectral residuals, sequence-to-sequence models and model distillation; Scikit-Lego for reconstruction through PCA or UMAP; Luminaire’s window density

model; Donut, a variational autoencoder for seasonal key performance indicators; rrcf, a robust random cut forest for streaming detection; banpei, based on Hotelling’s theory; STUMPY, implementing matrix-profile algorithms including STUMP, FLUSS and FLOSS; and PySAD for streaming outlier detection.

11.2 Change-point detection

A **change point**, or tipping point, marks a moment at which the behaviour of the observed system changes significantly—an abrupt and significant transition between states of the generating process. The trend can change suddenly, and a change point signals where. In climate modelling, candidate tipping points around 2010 and the early 2020s represent major shifts in greenhouse-gas emission trends and hence in projected warming scenarios. A more immediate illustration: running a quarter mile, walking for ten minutes, then running another quarter mile produces a heart-rate series showing a cluster of high rates, a cluster of low rates, and a return to high rates, with abrupt changes marking the transitions in activity. Applications include speech recognition, where word and sentence boundaries must be found; image analysis and video surveillance; fitness, where human activities are segmented from wearable motion sensors; and finance, where trend changes may signal a shift between bear and bull markets—the formal counterpart of technical chart-pattern analysis.

Under the assumption of *offline* segmentation, in which the whole signal is available (the framework adapts readily to the online setting), the goal is to recover both the number of change points K^* and their times $T^* = (t_1^*, \dots, t_{K^*}^*)$ by solving

$$(\hat{t}_1, \dots, \hat{t}_K) = \underset{(t_1, \dots, t_K)}{\operatorname{argmin}} \sum_{k=0}^K c(x[t_k : t_{k+1}]),$$

with the conventions $t_0 = 0$, $t_{K+1} = N$, and $a:b = [a, a+1, \dots, b-1]$. Every algorithm in this family is a choice of three components: a **cost function**, a **search method**, and a **constraint**.

11.2.1 Cost functions

The cost $c(\cdot)$ measures the homogeneity of a segment, and choosing it determines what counts as a change. The most common costs derive from parametric probabilistic models, in which change points are defined as changes in the parameters of a probability density function. Given a parametric family of densities $f(\cdot | \theta)$ with $\theta \in \Theta$, the maximum-likelihood cost is

$$c_{\text{ML}}(x[a:b]) = -\sup_{\theta} \sum_{n=a+1}^b \log f(x[n] | \theta),$$

corresponding to the assumption that within a regime the samples are i.i.d. according to that density, with parameters estimated per segment by maximum likelihood. The model adapts to several situations, and its special cases give the standard costs.

Change in mean. The L_2 cost is the Gaussian case with fixed variance, and by far the most popular:

$$c_{L_2}(x[a:b]) = \sum_{n=a+1}^b \|x[n] - \mu_{a:b}\|_2^2,$$

with $\mu_{a:b}$ the empirical mean of the segment.

Change in mean and variance. When both change, the cost becomes

$$c_{\Sigma}(x[a:b]) = (b-a) \log \sigma_{a:b}^2 + \frac{1}{\sigma_{a:b}^2} \sum_{n=a+1}^b \|x[n] - \mu_{a:b}\|_2^2,$$

the Gaussian case with unknown mean and variance. Replacing the variance by a covariance matrix extends this to multivariate series, in which case changes in the *correlation between dimensions* become detectable.

Change in slope and intercept. Piecewise linear regression handles this in general:

$$c_{\text{linear}}(x[a:b]) = \min_{\alpha} \sum_{n=a+1}^b \left\| x[n] - \sum_{i=1}^M \alpha_i \beta_i[n] \right\|_2^2,$$

where $\beta_1[n], \dots, \beta_M[n]$ are covariate functions and the change sought is in the regression parameters. Taking $\beta_1[n] = 1$ and $\beta_2[n] = n$ detects changes in slope and intercept; other choices detect changes in trend, seasonality and related structure.

Rank-based costs. To remove the need for a parametric model, one works on ranks instead of the signal. Define $r[n]$ as the number of indices i with $x[i] < x[n]$ —the rank of sample n within the series. Then

$$c_{\text{rank}}(x[a:b]) = \sum_{n=a+1}^b \|r[n] - \mu_{a:b}^r\|_2^2$$

detects changes in the mean or variance of the rank signal. It is robust and invariant with respect to amplitude changes.

Least absolute deviation. A simpler robust option, estimating a shift in the central point of a distribution:

$$c(x_I) = \sum_{t \in I} |x_t - \hat{x}|,$$

where I indexes a subsection of the series and $\hat{x}$ is its central point.

Running the same signal through different costs at the same K produces visibly different segmentations. The cost function, not the search method, encodes what "change" means.

11.2.2 Search methods

With K fixed, the objective can be solved exactly or approximately. Writing $V(T, x) = \sum_{k=0}^K c(x[t_k:t_{k+1}])$, the problem is recursive:

$$\min_{|T|=K} V(T, x) = \min_{t \leq N-K} \left[c(x[0:t]) + \min_{|T|=K-1} V(T, x[t:N]) \right],$$

which is solved by **dynamic programming**—the same structure that underlies DTW—in two steps: computation of the cumulative costs, then determination of the change points by backtracking. The solution is optimal, at a complexity of $O(KN^2)$, which is prohibitive for long series.

The **sliding-window** approximation trades optimality for speed by reducing the task to repeated single-change detection, which is far easier. On a window of length $2w$ centred at sample n , the **discrepancy function** is

$$d[n] = c(x[n-w:n+w]) - c(x[n:n+w]) - c(x[n-w:n]),$$

comparing the homogeneity of the whole window against the homogeneities of its right and left halves. If a change point occurs at n and the window length is well adapted, both subsegments will be homogeneous—small costs—while the whole window will be heterogeneous, giving a large cost. Large values of $d[n]$ therefore indicate a likely change point, and detection reduces to finding the K largest peaks, at a complexity of $O(N)$.

Choosing w involves a real tension. It should correspond to the smallest length of stationarity, since the discrepancy function is meaningful only when the two subsegments are internally homogeneous, and it is necessarily smaller than the shortest regime. Yet it must also be large enough that each half contains sufficient samples to estimate homogeneity properly—the same trade-off as choosing a sample size for a two-sample statistical test.

Binary segmentation is the greedy window-based alternative. It finds the single split minimising the total cost,

$$\hat{t} := \operatorname{argmin}_{1 \leq t < T-1} c(x_{0...t}) + c(x_{t...T}),$$

then recurses on the resulting segments. The best split is the one making both parts internally consistent; plotting total cost against candidate split point shows a clear minimum at the change. Bottom-up methods invert the procedure.

11.2.3 Determining the number of change points

Everything above assumes K known, and in practice it rarely is. Worse, the total cost $V(T, x)$ decreases monotonically as K increases, so the objective alone cannot select it. Three solutions exist.

The *heuristic* approach tests $K = 1, \dots, K_{\max}$, computes the sum of costs for each, and estimates the optimal value by locating an elbow in the curve of $V(T, x)$ against K .

The *penalised* formulation estimates times and number jointly:

$$(\hat{t}_1, \dots, \hat{t}_K) = \operatorname{argmin}_{(t_1, \dots, t_K), K} \sum_{k=0}^K c(x[t_k:t_{k+1}]) + \beta K,$$

where β penalises the introduction of a new change point: an additional change point must reduce the sum of costs by at least β to earn its place. The same problem recurs elsewhere in model selection—the order of an AR model, the number of atoms in a dictionary—where higher order always improves reconstruction and something must arrest it.

Counter-intuitively, this joint problem is *easier* than the fixed- K problem, because it admits a pruning rule.

Given times $t < s < N$, if

$$\begin{aligned} \min_T [V(T, x[0:t]) + \beta|T|] + c(x[t:s]) \\ \geq \min_T [V(T, x[0:s]) + \beta|T|], \end{aligned}$$

then t cannot be the last change point prior to N and may be discarded permanently. The argument adds $c(x[s:N]) + \beta$ to both sides and uses the fact that introducing a breakpoint never increases the sum of costs, $c(x[t:s]) + c(x[s:N]) \leq c(x[t:N])$, to conclude that s dominates t . Since most candidate times satisfy the criterion, the speed-up is considerable. This is the **PELT** algorithm—Pruned Exact Linear Time—which is *exact* rather than approximate and runs in $O(N)$ under the assumption that regime lengths are drawn from a uniform distribution. It is accordingly the method of choice: an optimal solution at linear cost.

Setting β remains the difficulty, since no explicit formula relates β to K . The information criteria of Section 8 supply defaults. In general,

$$\text{BIC} = k \log N - 2 \log \hat{L}, \quad \text{AIC} = 2k - 2 \log \hat{L},$$

with k the number of parameters, N the number of samples and $\hat{L}$ the maximum of the likelihood function. Specialised to change-point detection with the L_2 cost, these yield

$$\beta = 4\sigma^2 \log N \quad (\text{BIC}), \quad \beta = 4\sigma^2 \quad (\text{AIC}),$$

so that the BIC penalty grows with the length of the signal and produces fewer change points on long records. Alternatively β can be *learned* from a collection of annotated signals $x^{(1)}, \dots, x^{(M)}$ with annotations $T_*^{(1)}, \dots, T_*^{(M)}$. Writing $V_\beta(T, x) = \sum_k c(x[t_k:t_{k+1}]) + \beta K$, the **excess penalised risk**

$$E(x^{(\ell)}, \beta) = V_\beta(T_*^{(\ell)}, x^{(\ell)}) - \min_T V_\beta(T, x^{(\ell)})$$

reflects how well the annotated segmentation can be approached at a given β . It is a convex function of β and so is easily optimised with standard solvers, giving

$$\beta_{\text{opt}} = \underset{\beta > 0}{\operatorname{argmin}} \frac{1}{M} \sum_{\ell=1}^M E(x^{(\ell)}, \beta).$$

Experimentally, only a few annotated examples are needed to identify an adequate range for β .

11.2.4 Anomalies and change points compared

The two problems are closely related. Change-point detection connects to segmentation, edge detection, event detection and anomaly detection, and similar techniques apply across all of them. One way to identify change points is precisely to threshold the scores produced by an anomaly detection algorithm, and from that perspective change points are identical to highly anomalous points, with anything above a chosen threshold corresponding to a change. What differs is the question being asked: anomaly detection asks whether a sample is unusual, while change-point detection asks where the statistical properties of the process transition between states—and must therefore also estimate how many such transitions occur.

11.3 Clustering and similarity

Cluster analysis finds meaningful groups of points or objects in a dataset on the basis of their similarity, such that points within a cluster resemble one another while differing from points in other clusters. It is harder for temporal data because each data point is itself a period of time—an ordered sequence—rather than a vector of independent attributes. The choice of distance measure strongly affects the quality of the result.

Euclidean distance compares two series point by point:

$$d(x, y) = \sqrt{\sum_{t=1}^T (x_t - y_t)^2}.$$

It works well when the series are aligned in time and fails otherwise: two curves of identical shape offset by a few steps register as far apart.

Manhattan distance uses absolute rather than squared differences,

$$d(x, y) = \sum_{t=1}^T |x_t - y_t|,$$

and is less sensitive to a single large deviation, which suits cases where extreme errors should not dominate. It shares the alignment and equal-length requirements.

Correlation distance measures whether two series move together, ignoring absolute scale:

$$d(x, y) = 1 - \operatorname{corr}(x, y).$$

Series of very different magnitude are treated as similar if their patterns rise and fall together. For $A = [1, 2, 3, 4]$ and $B = [10, 20, 30, 40]$ the Euclidean distance is large while the correlation distance is near zero, because the shape is identical.

Dynamic time warping computes an optimal match by warping the time axis to find the lowest possible total distance, so it accommodates both temporal shifts and unequal lengths, at $O(T^2)$ cost.

Cluster quality is assessed by the **silhouette score**, which measures how similar a point is to its own cluster relative to other clusters:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))},$$

where $a(i)$ is the average distance from point i to points in the same cluster and $b(i)$ the average distance to points in the nearest other cluster. Values near +1 indicate very good clustering, near 0 overlapping clusters, and near −1 outright misassignment. One methodological point is easy to get wrong: the silhouette must be computed with the *same* distance measure used for the clustering. Clustering with DTW and evaluating with ordinary Euclidean distance measures nothing coherent.

11.4 Evaluating event detection

Pattern recognition, change-point detection and anomaly detection are all **event detection** tasks, and benchmarking them requires an annotated dataset in

which the events of interest have been highlighted, together with appropriate metrics. When events consist of single points, standard precision and recall suffice:

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN},$$

with TP , FP and FN the numbers of true positives, false positives and false negatives. Both lie in $[0, 1]$, and a precision–recall curve compares methods across operating points.

Events are frequently *ranges*, however, and point metrics then misrepresent performance in at least three ways. A detection may be correct but delayed, which is unacceptable in industrial monitoring even though the event was found. The event may be detected but its duration poorly estimated. Or two annotated events may be merged into a single predicted range.

Four principles address this. *Existence*: catching the existence of the event at all, even by predicting a single point, may be valuable in itself. *Size*: the larger the correctly predicted portion, the higher the recall should be. *Position*: in some applications the relative position of the correctly predicted portion matters as well as its size. *Cardinality*: detecting an event with one prediction range is more valuable than doing so with several fragmented ranges.

Given predicted intervals $P = \{P_1, \dots, P_{N_P}\}$ and real intervals $R = \{R_1, \dots, R_{N_R}\}$, recall and precision average per-event scores:

$$\begin{aligned} \text{recall} &= \frac{1}{N_R} \sum_{i=1}^{N_R} \text{recall}(R_i, P), \\ \text{precision} &= \frac{1}{N_P} \sum_{i=1}^{N_P} \text{precision}(R, P_i). \end{aligned}$$

Each term is a weighted combination of the components above. Writing Ex , SP and Cd for existence, size-and-position and cardinality, existence—used for recall only—is

$$\text{Ex}(R_i, P) = \begin{cases} 1 & \text{if } \sum_{j=1}^{N_P} |R_i \cap P_j| \geq 1 \\ 0 & \text{otherwise,} \end{cases}$$

size and position together are

$$\text{SP}(R_i, P) = \sum_{j=1}^{N_P} w(R_i, R_i \cap P_j),$$

where $w(A, B)$ is an overlap score in $[0, 1]$ between the detected intersection and the desirable portion of R_i —and may deliberately introduce a bias, if for instance the event should be caught in advance, or its middle portion matters most. Cardinality is

$$\text{Cd}(R_i, P) = \begin{cases} 1 & \text{if } R_i \text{ overlaps } \leq 1 \text{ interval} \\ \gamma(R_i, P) & \text{otherwise,} \end{cases}$$

with γ a penalty function. These combine as

$$\begin{aligned} \text{recall}(R_i, P) &= \alpha \text{Ex}(R_i, P) \\ &\quad + (1 - \alpha) \text{Cd}(R_i, P) \text{SP}(R_i, P), \end{aligned}$$

$$\text{precision}(R, P_i) = \text{Cd}(R, P_i) \text{SP}(R, P_i).$$

Different w and γ may be used for precision and for recall. The framework generalises the standard definitions: if all events in R and P are single points, $\alpha = 0$, $\gamma \equiv 1$, and w is the percentage of common points, it reduces exactly to ordinary precision and recall.

A simpler alternative is the **intersection over union** between a detected and a true segment,

$$\text{IoU} = \frac{|P_i \cap R_j|}{|P_i \cup R_j|},$$

thresholded at a chosen value such as 25%, 50% or 75%. Reporting performance across several thresholds conveys the accuracy of a detection method far better than any single number.

12 Deep learning for sequences

Deep sequence models are best approached through one organising idea, the **encoder–decoder paradigm**: the encoder consumes the history and retains the information required for the decoder to generate the forecast. Every architecture below can occupy either role, and that interchangeability is what the paradigm buys.

12.1 Feed-forward networks

A **feed-forward network (FFN)** is trained by forward and backward propagation with gradient descent on a loss function, a learning rate controlling step size, ReLU, tanh or sigmoid activations in the hidden layers, and softmax at the output for classification. In forecasting, time is embedded and the problem converted into a regression problem before the data reaches the network. An FFN can then serve as an encoder—used exactly as the machine-learning models of Section 10 are used—or as a decoder, mapping the latent vector produced by the encoder to the output.

A monthly passenger-volume series illustrates how much structure can be added without leaving this framework. A naive network improves substantially through four changes: adding an **embedding** of the month feature—a learned dense vector per category, richer than a one-hot indicator; treating the year as a linear predictor; feeding the previous month’s value as an input; and rescaling predictions by the standard deviation of the training data. The last step inverts standardisation. Since

$$z = \frac{x - \mu}{\sigma} \iff x = z\sigma + \mu,$$

the prediction rule becomes

$$\hat{v}_t = p\sigma + v_{t-1},$$

so that the network predicts a standardised increment p , which is rescaled and added to the previous value. Predicting a scaled change rather than an absolute level is a small design decision with a large effect on trending series.

12.2 Recurrent networks and LSTMs

A **recurrent neural network (RNN)** processes a sequence step by step while maintaining a **hidden state**. The outputs and hidden state encode the information

contained in the sequence and can be used for further processing—predicting the next step through a decoder, for instance—and an RNN block can itself act as the decoder, consuming an encoded representation and producing outputs. This flexibility permits a wide variety of input/output arrangements, of which two are standard: **many-to-one**, with many inputs and a single output, as in single-step forecasting or sequence classification; and **many-to-many**, with many inputs and many outputs, as in multi-step forecasting.

The parameters of a standard RNN layer make the design choices explicit. `input_size` is the number of expected features: 1 when only the history of the series is used, greater when additional features accompany it. `hidden_size` is the dimension of the hidden state, and thereby fixes the size of the input-to-hidden and hidden-to-hidden matrices. `num_layers` is the number of RNNs stacked on top of one another. `nonlinearity` selects tanh, the originally proposed choice, or ReLU. `bias` determines whether bias terms enter the update equations. `batch_first` chooses between the input layouts (batch size, sequence length, features) and (sequence length, batch size, features)—a persistent source of silent errors. `dropout`, if non-zero, applies a dropout layer to the outputs of every layer except the last; dropout is a regularisation technique in which randomly selected neurons are ignored during training. `bidirectional` runs the sequence in both directions, which is legitimate for classification but violates causality for forecasting.

Long short-term memory (LSTM) networks address the difficulty plain RNNs have in retaining information over long spans. An LSTM cell maintains a **cell state** running along the sequence, regulated by three gates. The **forget gate** decides what to discard from the cell state; the **input gate** decides what new information to store; and the **output gate** decides what to emit as the hidden state. Sigmoid activations produce the gating values, which lie in $[0, 1]$ and therefore express "how much to let through", while tanh produces candidate values, and the cell state is updated by pointwise multiplication and pointwise addition. Because the cell state is modified additively rather than by repeated matrix multiplication, gradients survive across far longer horizons. An LSTM layer returns both hidden states and cell states, and otherwise takes the same parameters as an RNN layer.

12.3 Convolutional networks

Convolutional neural networks (CNNs) process data arranged on a grid, which may be 2D as in an image, 1D as in a time series, or 3D as in LIDAR data. A kernel slides over the input computing local weighted sums, so that in the 1D case it acts as a learned detector of local temporal patterns.

Three parameters govern the geometry. **Padding** adds values at the edges of the sequence—`valid` for none, `same` to keep the output length equal to the input length—with the fill values determined by a padding mode, commonly `zeros` or `replicate`, and also `reflect` or `circular`. **Stride** sets how far the kernel slides at each step. **Dilation** sets how many elements are skipped between kernel taps, expanding the recep-

tive field without adding parameters. The output length follows

$$O = \frac{L + p_l + p_r - d \times (k - 1) - 1}{s} + 1,$$

where L is the input length, p_l and p_r the left and right padding, k the kernel size, s the stride and d the dilation.

In the standard interface, `in_channels` is the number of input features—usually 1 for a single series, greater with extra features, and equal to the previous layer's `out_channels` in later layers—while `out_channels` is the number of filters, each learning a different pattern from the input. `groups` controls the connections between input and output channels and is normally left at 1, and `bias` adds a learnable bias term.

The **dilated causal convolutional network** combines two ideas. *Causal* padding ensures that each output depends only on current and past inputs, which is mandatory for forecasting. *Dilation* increasing geometrically across layers—1, 2, 4, 8, 16, 32, 64 in a typical stack—makes the receptive field grow exponentially with depth, so that a modest number of layers observes a long history. Residual connections, which add each block's input back to its output, together with skip connections and dropout, keep the stack trainable. The architecture attains long memory without recurrence, and therefore without sequential computation.

12.4 Attention and transformers

Attention removes the information bottleneck of the plain encoder-decoder design, in which the entire history must be compressed into a single fixed-size context vector. Instead the decoder computes a weighted combination of *all* encoder hidden states $h_1, \dots, h_T$, the weights α_i reflecting how relevant each input step is to the current output step. The context vector therefore differs at every decoding step.

Transformers dispense with recurrence entirely in favour of stacked self-attention. Because attention is order-agnostic, position must be injected explicitly through **positional encoding**:

$$p_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right),$$

$$p_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right),$$

where `pos` is the position in the sequence, i indexes the embedding dimension and d_{model} is the model width. Sinusoids of geometrically varying wavelength allow the model to represent both fine and coarse positional distinctions and to generalise to positions unseen during training.

The architecture stacks N encoder blocks—multi-head attention followed by a feed-forward layer, each wrapped in an add-and-norm residual connection—and N decoder blocks, comprising masked multi-head attention, encoder-decoder attention and a feed-forward layer, with a linear projection and softmax at the output. **Masking** supplies causality: a triangular mask applied to the QK^T attention matrix prevents position t from attending to any position after t . In practice this is

a lower-triangular boolean matrix, converted so that masked entries become $-\infty$ before the softmax and consequently receive zero weight. A forecasting implementation projects the input to d_{model} , adds the positional encoding, runs the masked transformer encoder, and decodes to the horizon.

12.5 Comparative behaviour

Multi-step forecasting results on a single univariate series are instructive:

Model	MAE	MSE	MASE	Bias
LSTM (teacher forcing)	0.1754	0.0912	1.3671	12.93%
Seq2Seq + dot attn.	0.1536	0.0717	1.1967	8.25%
Transformer (FF dec.)	0.1949	0.1104	1.5188	16.24%

Attention improves on the plain LSTM across every metric, yet the transformer—the largest and most modern of the three—performs worst. This is not an anomaly to be explained away; it is the same lesson the Theta method teaches in the classical setting. On a single series of modest length, model capacity is not the binding constraint, and additional parameters cost more in variance than they return in expressiveness. Note also that all three MASE values exceed 1, meaning that none of these models beats the naive baseline on this problem—a reminder that the informative comparison is against a baseline, not against other neural networks.

13 Synthesis

Time-series analysis rests on one substantive assumption and one methodological one.

The substantive assumption is *stationarity*. The classical apparatus—ACF and PACF, ARMA and its extensions, the Fourier transform, the cost functions of change-point detection—is valid only where the distribution is invariant in time. Almost every technique in the field is therefore either a means of achieving stationarity (differencing, decomposition, framing, power transformations), a means of testing for it (ADF, KPSS, inspection of the ACF), or a means of exploiting the places where it fails (change-point detection, GARCH, drift-aware validation).

The methodological assumption is *causality*. Features, validation splits, decomposition windows and network architectures must all respect the prediction time, and each family of methods has its own characteristic way of violating it: centred moving averages, k -fold cross-validation, bidirectional recurrent layers, unmasked attention.

Around this spine run three threads. The *decomposition thread* is the recurring claim that a series is trend plus seasonality plus residual. It appears as an exploratory tool, as preprocessing before outlier detection and imputation, as the mechanism of Holt–Winters, as the design of hybrid frameworks such as Silverkite, and as the basis of model-based anomaly detection. The *representation thread* is the problem of turning variable-length ordered sequences into something a general-purpose learner can consume: calendar features, lag features, ROCKET’s maximum-and-PPV summaries, shapelet distances, automated statistical descriptors, and the hidden state of a recurrent network are all answers to one question, differing only in how much is designed and how much

is learned. The *similarity thread* is the observation that Euclidean, Manhattan, correlation and DTW distances underpin nearest-neighbour classification, clustering, the matrix profile and hence anomaly detection—so that the choice of distance quietly determines the results of several apparently unrelated methods.

The three modelling families answer different constraints rather than forming a chronological progression. Classical models are interpretable, data-efficient and correct when their assumptions hold. Machine-learning models handle non-linearity, exogenous variables and large collections of related series, at the cost of demanding feature engineering. Deep models learn the representation as well as the mapping, and repay that flexibility only where there is enough data to estimate it. The empirical evidence points consistently in one direction: the Theta method winning competitions, attention outperforming a transformer, scaled errors above 1 against a naive baseline. Complexity must be earned. Information criteria formalise this within the classical family, penalised change-point detection formalises it for segmentation, and regularisation formalises it for linear models—three statements of the single idea that fit must be traded against the number of parameters.

Evaluation binds all of it together, and it is where temporal analysis departs most sharply from generic supervised learning. Errors must be measured on genuinely unseen future data through walk-forward validation. The choice among squared, absolute, percentage and scaled error metrics encodes what kind of mistake matters. And for detection tasks, point-based precision and recall must give way to range-aware metrics accounting for existence, size, position and cardinality. A forecast or a detection is only as trustworthy as the protocol that measured it.